

System Design Note

Vehicle ECU Simulation Hackathon

C9

Abstract

This document outlines the software architecture and system design for the Vehicle ECU Simulation project. It details the cyclic scheduler flow, the specific responsibilities of each software module, the logic governing safe-state transitions, and the system's fault handling and escalation strategies.

1 File Structure Snapshot

The project follows a strict modular structure to separate interfaces and implementations.

```
/project_root
|-- /src
|   |-- main.c
|   |-- input.c
|   |-- mode.c
|   |-- control.c
|   |-- fault.c
|   |-- state.c
|   |-- log.c
|-- /include
|   |-- types.h
|   |-- input.h
|   |-- mode.h
|   |-- control.h
|   |-- fault.h
|   |-- state.h
|   |-- log.h
```

2 Scheduler Flow

The ECU execution model relies on a deterministic, continuous loop scheduler. During every cycle, tasks are executed in the following strict order to prevent race conditions and ensure accurate state evaluation:

- 1. Read Inputs:** Collects and validates speed, temperature, gear, and requested mode.
- 2. Update Vehicle Mode:** Checks for legal mode transitions and updates the operational mode (or forces FAULT mode).

3. **Run Control Logic:** Evaluates speed, temperature, and gear validity against defined thresholds.
4. **Update Fault Manager:** Sets/clears fault flags and increments counters based on control logic outputs.
5. **Evaluate System State:** Determines if the vehicle should be in NORMAL, DEGRADED, or SAFE state based on escalation rules.
6. **Generate Logs:** Prints structured summaries, warnings, and state transitions to the console.

Note: Altering this execution order could result in faults being evaluated on stale data, leading to delayed state transitions and potential system hazards.

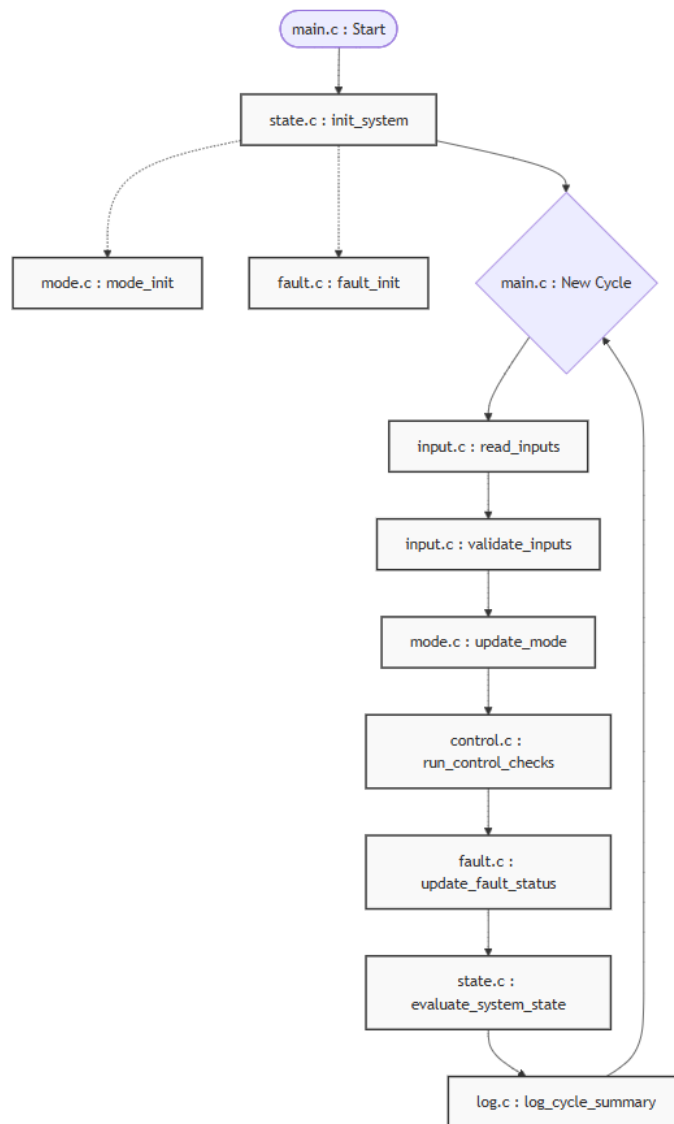


Figure 1: Cyclic Scheduler Execution Flow

3 Module Responsibilities

- **main.c:** Contains the primary execution loop and integrates all subsystems.
- **input (h/c):** Handles raw input ingestion and normalizes data to safe bounds.
- **mode (h/c):** Manages the mode state machine (OFF, ACC, IGNITION_ON, FAULT) and prevents illegal jumps.
- **control (h/c):** Contains the core business logic (evaluating overspeed, overheating, and gear correctness).
- **fault (h/c):** Centralized fault tracker utilizing bitwise flags and persistence counters.
- **state (h/c):** Safe-state manager determining operational safety (NORMAL, DEGRADED, SAFE).
- **log (h/c):** Standardized logging utility for system introspection.
- **types.h:** Shared repository for centralized structs, enums, and macro definitions.

4 State Transition Logic

The safe-state manager transitions the system deterministically based on active faults:

- **NORMAL:** Active when no faults are present in the system.
- **DEGRADED:** Triggered when the system detects one major active fault (e.g., initial overspeed) or repeated minor warnings.
- **SAFE:** Triggered when two or more critical faults are active persistently. Once entering SAFE mode, the system locks and requires a specific reset condition to recover.

5 Fault Handling Strategy

Faults are evaluated via bitwise operations allowing multiple concurrent faults to be recorded efficiently.

- **Priority Protocol:** 1. Critical Overheat → 2. Invalid Mode/Gear → 3. Overspeed → 4. High Temperature.
- **Tracking:** Each specific fault possesses a dedicated counter. If a fault persists across multiple scheduler cycles, it is deemed persistent, triggering further escalation to the state manager.
- **Recovery:** If inputs return to normal operating bounds, warnings may clear, but critical faults require defined recovery procedures as documented in the codebase.